UNIVERSITÀ
DEGLI STUDI
FIRENZE

Parallel Programming

Prof. Marco Bertini

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Shared memory: OpenMP

Implicit threads: motivations

- Implicit threading frameworks and libraries take care of much of the minutiae needed to create, manage, and (to some extent) synchronize threads.

- They greatly **simplify concurrent programming** hiding details, **at the expense of expressiveness and flexibility**.

- … in this lecture we are going to see an example of implicit threading framework for C/C++ and Fortran: OpenMP

Implicit threads: motivations

- OpenMP **does not require restructuring** the serial program (or anyway greatly reduces restructuring). The use only needs to add compiler directives to reconstruct the serial program into a parallel one.

- If no OpenMP library function is used, i.e. only compiler directives are used, then the structure of the program remains the same !

OpenMP

- OpenMP is an API for shared-memory parallel programming.

- The “MP” in OpenMP stands for “multiprocessing,” a term that is synonymous with shared-memory parallel computing.

- OpenMP is designed for systems in which each thread or process can potentially have access to all available memory. When using OpenMP, we view our system as a collection of cores or CPUs, all of which have access to main memory,

- It’s not just a library: it’s a set of compiler directives (must be supported by the compiler), library routines, and environment variables.

OpenMP

- Compiler directives: instruct the compiler on how to parallelize the code.
  - This is the core of OpenMP.
- Runtime library functions: modify and check the number of threads and check how may processors there are in the multiprocessor system.
- Environment variables to alter the execution of OpenMP applications (e.g. the number of max. threads to use).

OpenMP and compilers

- OpenMP has to be supported by the compiler.
- Modern C/C++ compilers support OpenMP:
  - Microsoft Visual Studio
  - Intel C/C++ for Windows and Linux
  - GCC
  - Clang
    - notable exception: older Apple Clang - when programming under OS X install clang or gcc, e.g. using Macports or Homebrew. Latest Apple clang supports OpenMP but do not provide the libomp library that must be installed using Macports/Homebrew.
  - Use the latest version of compilers to use the latest standard. Typically Clang was lagging behind gcc but now it’s almost on parity.

OpenMP and compilers

- A compiler that support OpenMP defines the symbol `_OPENMP`

- OpenMP is an active standard: check what version is supported by the compiler.
  - E.g. using `task` directive require support for OpenMP 3.0. Using SIMD vectorization requires OpenMP 4.0.

- Current version is 6.0 (released in Nov. 2024). Major releases are every 5 years.

OpenMP and compilers

- GCC 15 supports all of OpenMP 4.5, most of 5.0, 5.1, and 5.2, and the first 6.0 features

- Clang defaults to OpenMP 5.0 semantics since release 11.0, support for OpenMP 5.0 is almost complete.

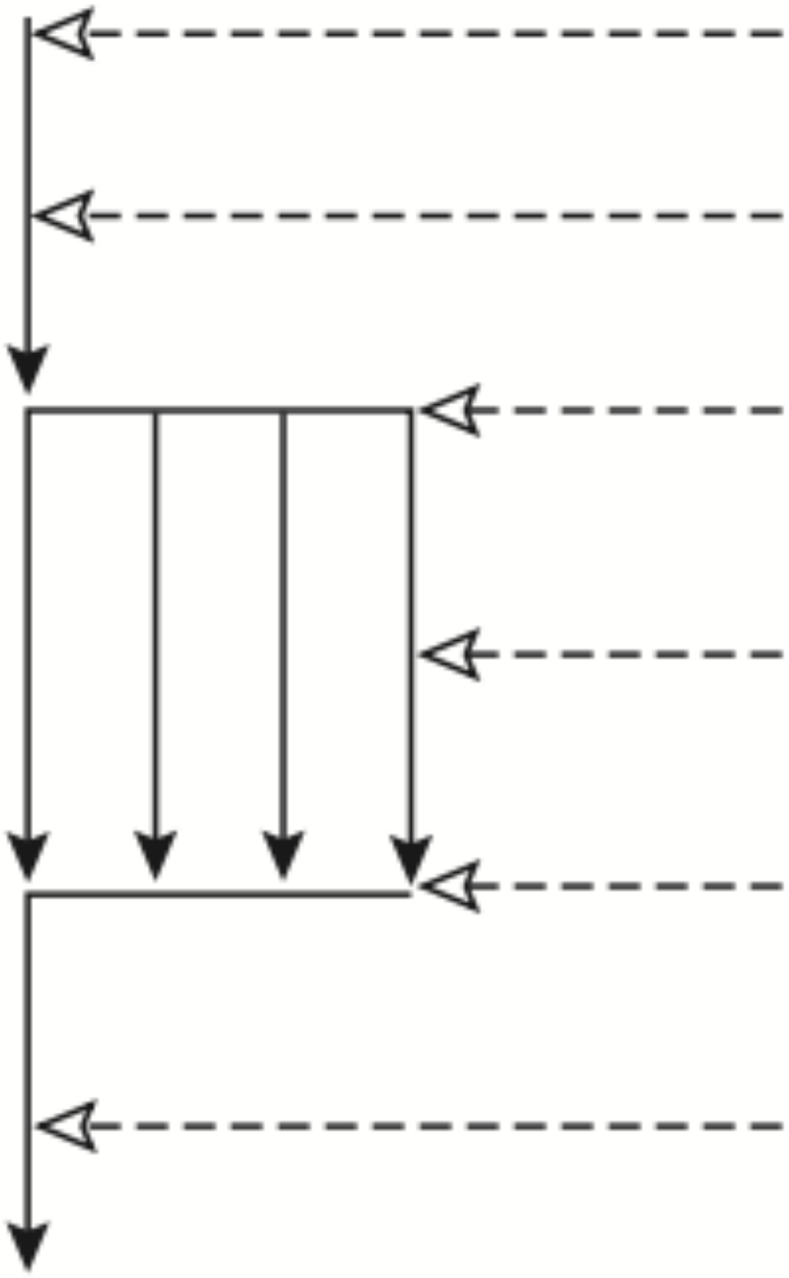
- he Microsoft Visual C/C++ compiler supports the OpenMP 2.0 standard with the `-openmp` switch.

  - Experimental support for more recent versions of the standard can be enabled by using the `-openmp:llvm` switch instead of the `-openmp` switch. Support for the SIMD directives from the OpenMP 4.0 standard is enabled with the `-openmp:experimental` switch

Thread model

- The OpenMP implicit threading follows the fork-join model:
  - the programmer indicates the regions of code that can be executed in parallel
  - when the compiled program founds those regions the main program continues as “main thread” and…
    - … multiple threads are forked, executing the parallel region.
    - Once a thread of this pool finishes it waits the end of the other threads (join)
    - When all threads have reached the “barrier” of joins the program continues executing the first instruction after the parallel region
- The parallel execution mode is SPMD, but it is possible to assign different tasks to different threads.

Thread model



- Main thread executes serial portion of the code.
- Main thread enters a routine with parallelized section.
- Main thread encounters parallel directive.
  Creates worker threads.
- Main and workers threads divide iterations of parallel code and execute them concurrently.
- Implicit barrier: Wait for all threads to finish their iterations.
- Main thread resumes execution after the parallelized code. Worker threads disappear.

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Memory consistency

Memory consistency model

- A memory consistency model is
  - A set of rules determining how to process memory operations from multiple processors
  - It is a contract between the programmer and the system
  - Determines what optimizations can be performed for correct programs
    - A compiler may rearrange the code
    - A CPU may rearrange the execution of operations

Memory consistency

- Memory consistency defines when updates by one thread become visible to others.

- Memory consistency models for parallel machines are based on the ordering enforced for memory accesses to different locations by a single processor.

  - Ensuring consistent views of memory is essential for correct parallel execution.

- We denote memory access ordering constraints by using the "→" (ordering) notation applied to reads (R), writes (W), and synchronizations (S).

Sequential Consistency

- It has been defined as the property that requires that:

  "... the result of any execution is the same as if the operations of all the processors were executed in some sequential order, and the operations of each individual processor appear in this sequence in the order specified by its program."

- It is as if all memory operations from different processors are serialized: a write to a variable does not have to be seen instantaneously, however, writes to variables by different processors have to be seen in the same order by all processors.

- It's a simple model for reasoning about parallel programs

Sequential Consistency

- Sequential consistency requires all memory accesses to complete in the same order as they occur in program execution:
  - R→R,
  - R→W,
  - W→R,
  - W→W.

  It also requires the effect of the accesses by all threads to be equivalent to performing them in some total (i.e., sequential) order.

- It is as if all memory operations from different processors are serialized: a write to a variable does not have to be seen instantaneously, however, writes to variables by different processors have to be seen in the same order by all processors.

- It's a simple model for reasoning about parallel programs

Sequential Consistency

- The result of a parallel program is always the same as the sequential program, irrespective of the statement interleaving that is a result of parallel execution

Sequential program	Parallel program 1	Parallel program n
`a = 5;`	`a = 5;`	`x = 1;`
`x = 1;`	`x = 1;`	`y = x+3;`
…	…	…
`y = x+3;`	`y = x+3;`	`a = 5;`
…	…	`z = x+y;`
`z = x+y;`	`z = x+y;`	…

Sequential Consistency

- The result of a parallel program is always the same as the sequential program, irrespective of the statement interleaving that is a result of parallel execution

Parallel programs have any order of permitted statement interleaving

Sequential program	Parallel program 1	Parallel program n
a = 5;	a = 5;	x = 1;
x = 1;	x = 1;	y = x+3;
…	…	…
y = x+3;	y = x+3;	a = 5;
…	…	z = x+y;
z = x+y;	z = x+y;	…

Relaxed consistency

- Sequential consistency is often considered difficult to maintain in modern multi-processors.
  The program order restriction prevents many important compiler optimizations that reorder program statements.

- Relaxed consistency models remove the ordering guarantees for certain reads and writes, but typically retain them around synchronizations.
  There are many types of relaxed consistency

Relaxed consistency

Relaxed consistency models have been successful because most memory operations in real parallel programs can proceed correctly without waiting for previous operations to complete.

Relaxed consistency semantics allows the overlapping of computation with memory access time, effectively hiding memory latency during program execution.

- Relaxed consistency models remove the ordering guarantees for certain reads and writes, but typically retain them around synchronizations. There are many types of relaxed consistency

Weak ordering

- Program order and atomicity is maintained only on a group of operations and not on all reads and writes. E.g. there's no need to show the operations of a critical section to all the processors before exiting it.

- Memory operations are *data operations* or *synchronization operations*

- Synchronization operations act like a fence:
  - All data operations before synch in program order must complete before synch is executed
  - All data operations after synch in program order must wait for synch to complete
  - Synchs are performed in program order

- Accesses to synchronization variables is sequentially consistent and any read or write should be performed only after previous synchronization operations have completed.

Weak ordering

- Program order and atomicity is maintained only on a group of operations and not on all reads and writes. E.g. there's no need to show the operations of a critical section to all the processors before exiting it.

Weak ordering prohibits overlapping a synchronization operation with any other shared memory operations of the same thread, while synchronization operations must be sequentially consistent with other synchronization operations.

Thus, the set of orderings guaranteed by weak ordering is the following:

- S→W,
- S→R,
- R→S,
- W→S,
- S→S

- Accesses to synchronization variables is sequentially consistent and any read or write should be performed only after previous synchronization operations have completed.

OpenMP memory consistency

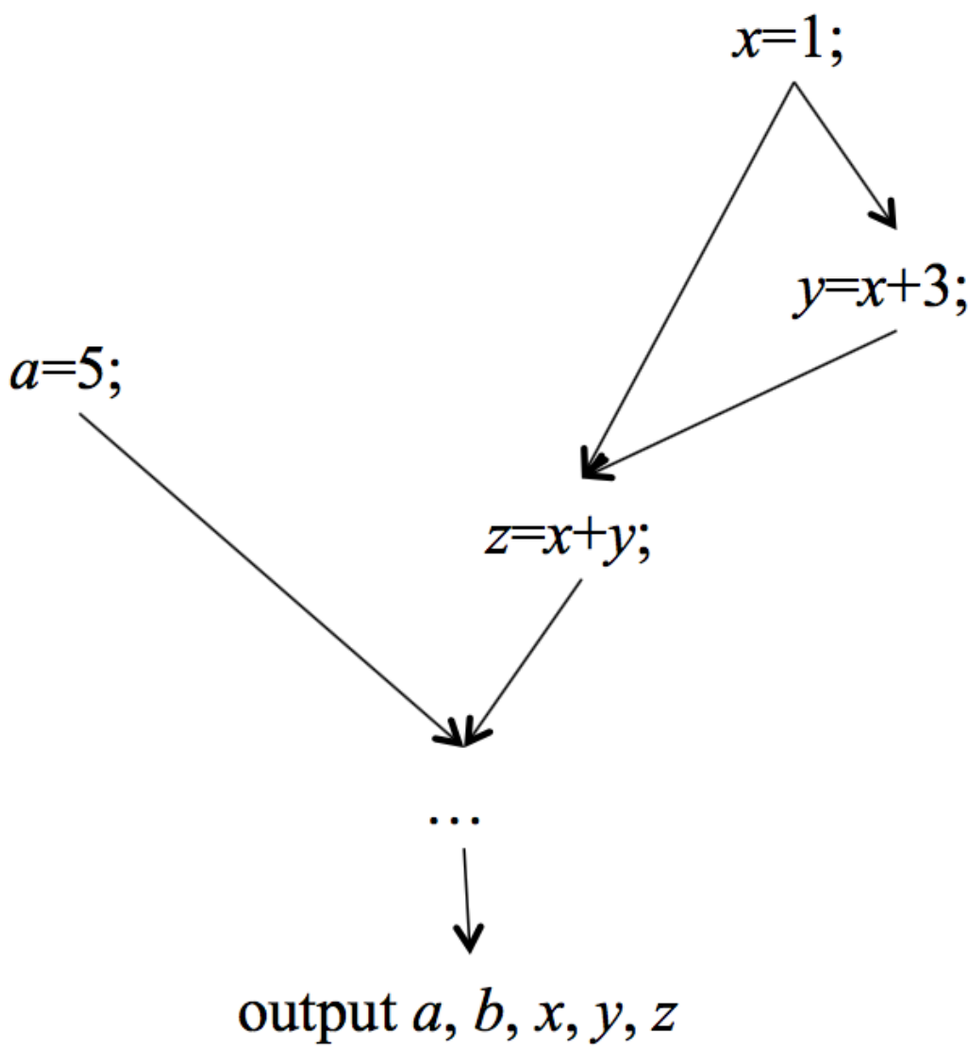
- OpenMP has a relaxed (weak) memory model
- Writes to shared data by one thread are not guaranteed to be visible to another thread till a synchronization point is reached.
- An OpenMP barrier or flush operation is required for the memory view of a thread to be communicated to other threads.
- OpenMP has both implicit and explicit memory synchronization points.

Data race

- Two memory accesses form a race if
  - From different threads, to same location, and at least one is a write
    - May execute consecutively in an SC global total order i.e., may execute “in parallel”
- Data-race-free-program = No data race in any SC execution
  - This is our goal !

Flow dependances

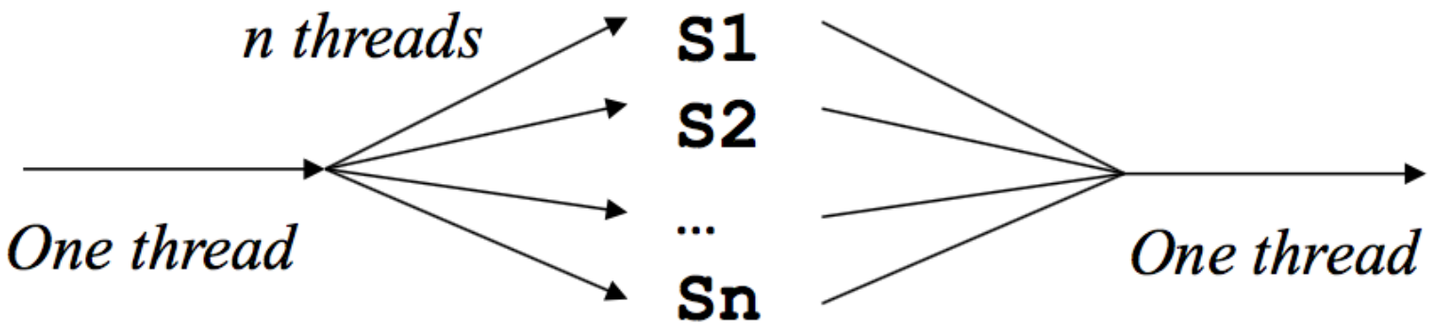
- Flow dependences determine the parallel execution schedule: each operation waits until operands are produced



Parallel programming constructs

- With the par construct the statements in the body are executed concurrently:

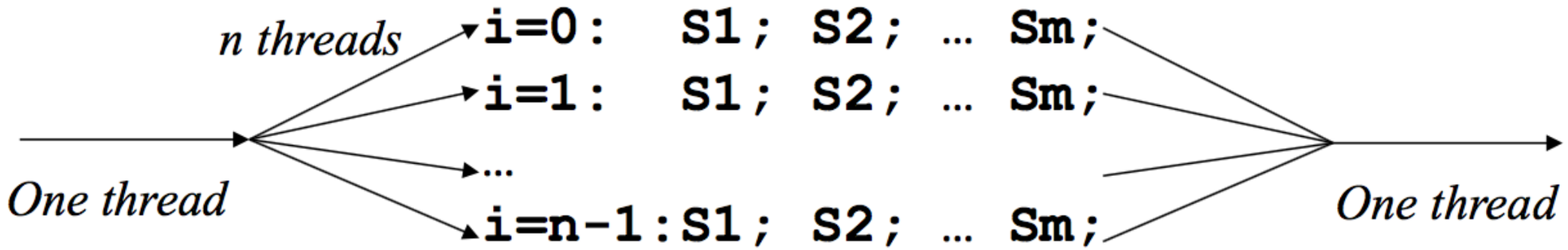
```
par {
   S1;
   S2;
    . . .
   Sn;
}
```



Parallel programming constructs

- With the `parfor` construct (also known as `forall`) the statements in the body are executed in serial order by n threads `i=0..n-1` in parallel

```
parfor (i=0; i<n; i++) {
   S1;
   S2;
   . . .
   Sn;
}
```



Data flow and dependance

- Considering data flow, we notice that each operation requires completion of operands first.
  If data dependancies are preserved the sequential consistency is guaranteed.

```
par {          x = 1;          x = 1;
  a = 5;       . . .           . . .
  x = 1;       par {           par {
}                a = 5;          a = 5;
. . .            y = x+3;        y = x+3;
y = x+3;     }                   z = x+y;
. . .        . . .           }
z = x+y;     z = x+y;        . . .
```

Data flow and dependance

- Considering data flow, we notice that each operation requires completion of operands first.
  If data dependancies are preserved the sequential consistency is guaranteed.

```
par {
   a = 5;
   x = 1;
}
. . .
y = x+3;
. . .
z = x+y;
```

```
x = 1;
. . .
par {
   a = 5;
   y = x+3;
}
. . .
z = x+y;
```

```
x = 1;
. . .
par {
   a = 5;
   y = x+3;
   z = x+y;
}
. . .
```

Bernstein's Conditions

- Processes cannot execute in parallel if one affects values used by the other. Nor can they execute in parallel if any subsequent process uses data affected by both, i.e. whose value might depend on the order of execution.
  - Bernstein's conditions are a simple test for deciding if statements or operations can be interchanged without modifying the program results.

- I_i is the set of memory locations read by process P_i
- O_j is the set of memory locations altered by process P_j
- Processes P_1 and P_2 can be executed concurrently if all of the following conditions are met
- $I_1 \cap O_2 = \varnothing$
- $I_2 \cap O_1 = \varnothing$
- $O_1 \cap O_2 = \varnothing$

These three constraints are too rigid if we want to share memory; however, if we provide some means to enforce a precedence among process sharing memory (e.g. semaphore, barrier, sync in general), then we can relax them.

Dependence analysis

- Dependence analysis performed by a compiler determines that Bernstein's conditions are not violated when optimizing and/or parallelizing a program

independent	RAW	WAR	WAW
P_1: A=x+y; P_2: B=x+z;	P_1: A=x+y; P_2: B=x+A;	P_1: A=x+B; P_2: B=x+z;	P_1: A=x+y; P_2: A=x+z;
$I_1 \cap O_2 = \varnothing$ $I_2 \cap O_1 = \varnothing$ $O_1 \cap O_2 = \varnothing$	$I_1 \cap O_2 = \varnothing$ $I_2 \cap O_1 = \{A\}$ $O_1 \cap O_2 = \varnothing$	$I_1 \cap O_2 = \{B\}$ $I_2 \cap O_1 = \varnothing$ $O_1 \cap O_2 = \varnothing$	$I_1 \cap O_2 = \varnothing$ $I_2 \cap O_1 = \varnothing$ $O_1 \cap O_2 = \{A\}$

Dependence analysis

- Dependence analysis performed by a compiler determines that Bernstein's conditions are not violated when optimizing and/or parallelizing a program

independent	RAW	WAR	WAW

par {
A=x+y;
B=x+z;
}
	P_1: A=x+y;
	P_2: B=x+A;
		P_1: A=x+B;
		P_2: B=x+z;
			P_1: A=x+y;
			P_2: A=x+z;

$I_1 \cap O_2 = \varnothing$ $I_2 \cap O_1 = \varnothing$ $O_1 \cap O_2 = \varnothing$	$I_1 \cap O_2 = \varnothing$ $I_2 \cap O_1 = \{A\}$ $O_1 \cap O_2 = \varnothing$	$I_1 \cap O_2 = \{B\}$ $I_2 \cap O_1 = \varnothing$ $O_1 \cap O_2 = \varnothing$	$I_1 \cap O_2 = \varnothing$ $I_2 \cap O_1 = \varnothing$ $O_1 \cap O_2 = \{A\}$

Bernstein's Conditions in Loops

- A parallel loop is valid when any ordering of its parallel body yields the same result

- ```
  parfor (I=4;I<7;I++)
      S1: A[I] = A[I-3]+B[I];
  ```

S1(4): A[4] = A[1]+B[4];
S1(5): A[5] = A[2]+B[5];
S1(6): A[6] = A[3]+B[6];

S1(5): A[5] = A[2]+B[5];
S1(4): A[4] = A[1]+B[4];
S1(6): A[6] = A[3]+B[6];

S1(4): A[4] = A[1]+B[4];
S1(6): A[6] = A[3]+B[6];
S1(5): A[5] = A[2]+B[5];

S1(6): A[6] = A[3]+B[6];
S1(5): A[5] = A[2]+B[5];
S1(4): A[4] = A[1]+B[4];

S1(6): A[6] = A[3]+B[6];
S1(4): A[4] = A[1]+B[4];
S1(5): A[5] = A[2]+B[5];

S1(5): A[5] = A[2]+B[5];
S1(6): A[6] = A[3]+B[6];
S1(4): A[4] = A[1]+B[4];
  ```

Loops and parallelization

```
for(i=1; i<10; i++)
  A[i] = A[i-1];// S1
```

The instances of S_1 can be executed only in sequential order because of flow dependance.
Do not parallelize.

```
for(i=1; i<10; i++)
  A[i] = A[i+1];// S1
```

can be parallelized with:

```
parfor(i=1;i<10;i++)
  B[i] = A[i+1]

parfor(i=1;i<10;i++)
  A[i] = B[i]
```

```
for(i=1; i<10; i++)
  A[i] = A[i-k];// S1
```

allows a degree k of parallelization

Bernstein’s conditions as a tool

- They are a systematic way to check if parallelization is safe, formalize intuition about dependencies
- Bernstein's conditions are sufficient for parallelization
  - If satisfied then it is definitely safe to parallelize
- OpenMP provides some tools that help to:
  - eliminate false dependencies (private clause)
  - Handle special patterns (reduction clause)
  - Synchronize for true sharing (atomic and critical directives)
  - Enforce consistency (flush and barrier directives)

UNIVERSITÀ
DEGLI STUDI
FIRENZE

Overview of OpenMP in C/C++

#pragma

- The #pragma directives offer a way for each compiler to offer machine- and operating system-specific features.

- If the compiler finds a pragma it does not recognize, it issues a warning, but compilation continues.

OpenMP directive

- The general form of an OpenMP directive is

```
#pragma omp directive [clauses [ ] ...]
```

- written in a single line. Use \ to split the directive on more lines, as usual for preprocessor directives, e.g.:

```
#pragma omp directive \
        [clauses [ ] ...]
```

- The clauses are optional and are different for different directives. Clauses are used to influence the behavior of a directive. Directives are case sensitive.

OpenMP directive

- The general form of an OpenMP directive is

  #p .]

- wi on
  m .g.:

```
#pragma omp directive \
        [clauses [ ] ...]
```

The clause list contains information about:

- conditional parallelization
- degree of concurrency
- data handling between serial/parallel code

- The clauses are optional and are different for different directives. Clauses are used to influence the behavior of a directive. Directives are case sensitive.

Parallel region

- Parallel regions are indicated using a `#pragma` pre-processor directive (`#pragma omp`)

- Regions are single instructions or block of code (indicated with { and })

- Example on how to parallelize a region of code:

```
#pragma omp parallel
{
    // code
}
```

Compiling with OpenMP

- Tell the compiler to use OpenMP with an appropriate switch
  - `-fopenmp`
  - Both clang and gcc require this switch
- You may need to tell the compiler where are the headers and libraries
  - e.g. on macOS you have to manually install them with Macports or Homebrew (libomp)
  - In earlier macOS Apple clang there was no support for OpenMP, it was required to install an additional clang

Compiling with OpenMP

- Tell the compiler to use OpenMP with an appropriate switch
  - `-fopenmp`
  - Both clang and gcc require this switch

```
set(CMAKE_CXX_COMPILER "/opt/local/bin/clang++-mp-3.9")
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -std=c++11 -fopenmp")
```

- e.g. on macOS you have to manually install them with Macports or Homebrew (libomp)
- In earlier macOS Apple clang there was no support for OpenMP, it was required to install an additional clang

Hello World with OpenMP

CMakeLists.txt × | main.cpp ×

```cpp
#include <omp.h> // for OpenMP library functions
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    }  /* All threads join master thread and disband */

}
```

Hello World with OpenMP

CMakeLists.txt × | main.cpp ×

```
#include <omp.h> // for OpenMP library functions
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    }  /* All threads join master thread and disband */

}
```

pragma directive

Hello World with OpenMP

include directive to use OpenMP library functions

pragma directive

CMakeLists.txt × main.cpp ×

```
#include <omp.h> // for OpenMP library functions
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    }  /* All threads join master thread and disband */

}
```

Hello World with OpenMP

CMakeLists.txt | main.cpp

```
#include <omp.h> // for OpenMP library functions
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    }  /* All threads join master thread and disband */

}
```

include directive to use OpenMP library functions

pragma directive

OpenMP library functions

OpenMP library functions

Hello World with OpenMP

CMakeLists.txt × | main.cpp ×

```cpp
#include <omp.h> // for OpenMP library functions
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    }  /* All threads join master thread and disband */

}
```

pragma directive

OpenMP library functions

OpenMP library functions

Hello World with OpenMP

CMakeLists.txt × | main.cpp ×

```cpp
#include <omp.h> // for OpenMP library functions
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    }  /* All threads join master thread and disband */

}
```

OpenMP library functions

OpenMP library functions

Hello World with OpenMP

CMakeLists.txt × | main.cpp ×

```cpp
#include <omp.h> // for OpenMP library functions
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    }  /* All threads join master thread and disband */

}
```

Hello World with OpenMP

CMakeLists.txt × | main.cpp ×

```cpp
#include <omp.h> // for OpenMP library functions
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char* argv[]) {
    int nthreads, tid;

/* Fork a team of threads giving them their own copies of variables */
#pragma omp parallel private(nthreads, tid)
    {

        /* Obtain thread number */
        tid = omp_get_thread_num();
        printf("Hello World from thread = %d\n", tid);

        /* Only master thread does this */
        if (tid == 0) {
            nthreads = omp_get_num_threads();
            printf("Number of threads = %d\n", nthreads);
        }

    }  /* All threads join master thread and disband */

}
```

Parallel code

Parallel loops

- Loops are good candidates for parallelization.
- OpenMP has a special construct: `parallel for` to divide loop blocks to the threads of the team
- It is possible to use **dynamic** scheduling (blocks are assigned to threads once they finish their previous block) or **static** scheduling (predefined # of threads each receiving only one block)
- Use a **chunk** argument to set the size of each block

Variables and threads

- By default variables in an OpenMP threaded program are shared between threads, **except**:
  - the loop index variable associated with a loop construct (each thread must have its own copy in order to correctly iterate through the assigned set of iterations);
  - variables declared within a parallel region or declared within a function that is called from within a parallel region;
  - any other variable that is placed on the thread's stack (e.g., function parameters).
- If you need a local copy use a `private` clause (as in the previous example). A local copy of the variables in the list will be allocated for each thread. The initial value of variables that are listed within the private clause will be undefined, and you must assign value to them before they are read within the region of use.

Synchronization

- A `critical` construct acts like a lock around a critical region. Only one thread may execute within a protected critical region at a time.

- An `atomic` construct ensures that statements will be executed in an atomic, uninterruptible manner.
  There is a restriction on which types of statements you can use with the atomic construct, and you can only protect a single statement.

- The `single` and `master` constructs will control execution of statements within a parallel region so that only one thread will execute those statements (as opposed to allowing only one thread at a time).

  - `single` will use the first thread that encounters the construct,
  - `master` will allow only the master thread (the thread that executes outside of the parallel regions) to execute the protected code.

Synchronization

- A `barrier` directive defines a point where each thread waits for all other threads to arrive. Once all the threads arrive at that point, they can all continue execution past the barrier.

- Each thread is therefore guaranteed that all the code before the barrier has been completed across all other threads.

Reduction

- A reduction clause may help in reducing synchronization issues due to critical sections: often there's need to apply the same operation on a shared variable… but we do not want to pay sync cost.

- A **reduction operator** is a **binary operation** (such as addition or multiplication) and a reduction is a computation that repeatedly applies the same reduction operator to a sequence of operands in order to get a single result. Furthermore, all of the intermediate results of the operation should be stored in the same variable: the reduction variable.

- OpenMP creates a private variable for each thread, and the run-time system stores each thread's result in this private variable. OpenMP also creates a critical section and the values stored in the private variables are added in this critical section.

UNIVERSITÀ
DEGLI STUDI
FIRENZE

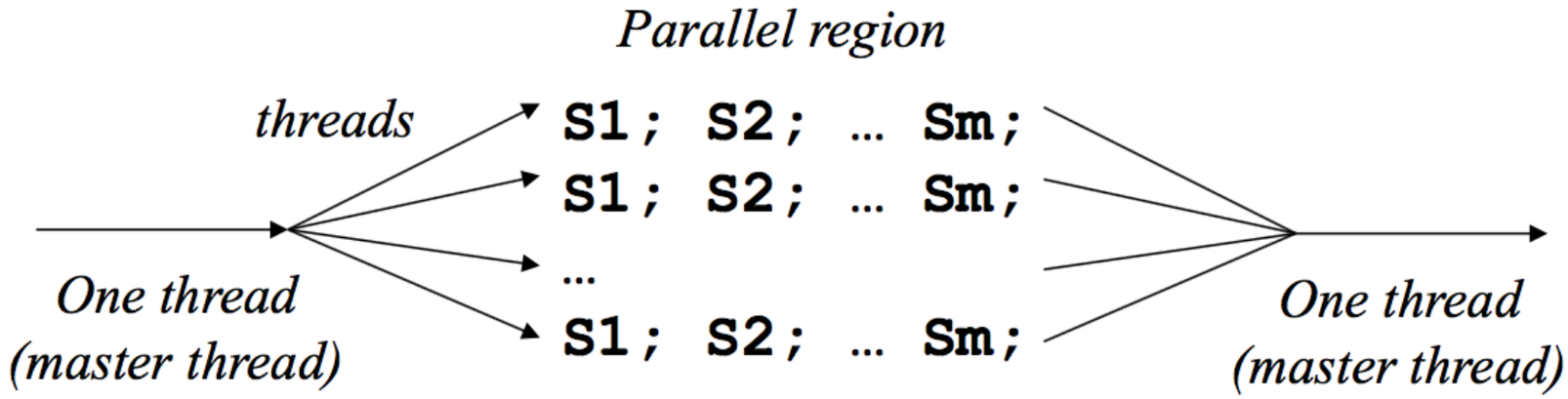
Overview of OpenMP directives

Parallel

- A team of threads all execute the body statements and joins when done

  - Note: is not the same as **par**

- ```
  #pragma omp parallel
  {
     S1;
     S2;
     ...
     Sm;
  }
  ```


  ```

`shared` / `private` clauses

- Specific clauses instruct wether variables are shared by thread or private, or set the default visibility of variables, e.g.:

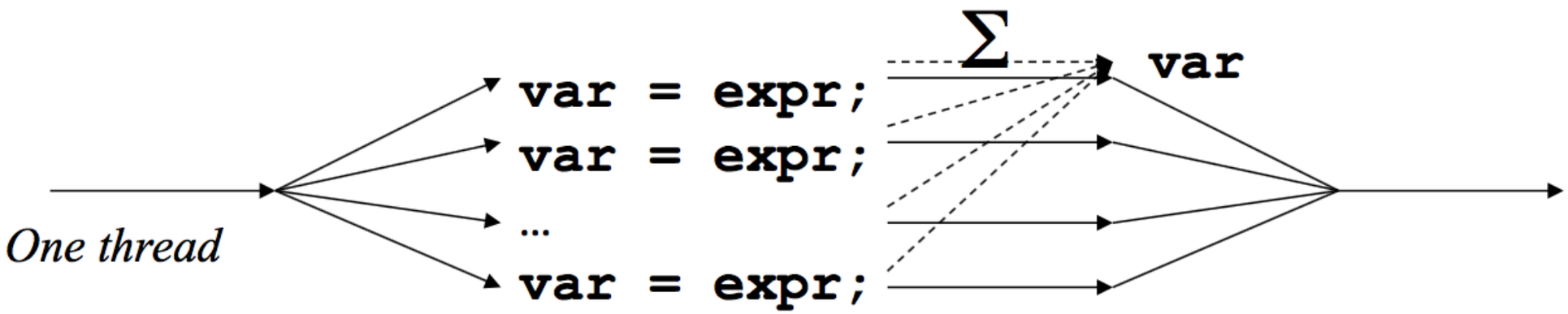
```
#pragma omp parallel default(none) \
    shared(var1, var2) private(var3)
```

- means that variables should not be shared by default, that `var1` and `var2` are shared by threads while `var3` is private in each thread

Parallel with reduction clause

- ```
  #pragma omp parallel reduction(+:var)
  {
     var = expr;
     ...
  }
  ... = var;
  ```

- Performs a global reduction operation over privatized variable(s) and assigns final value to master's private variable(s) or to the shared variable(s) when shared

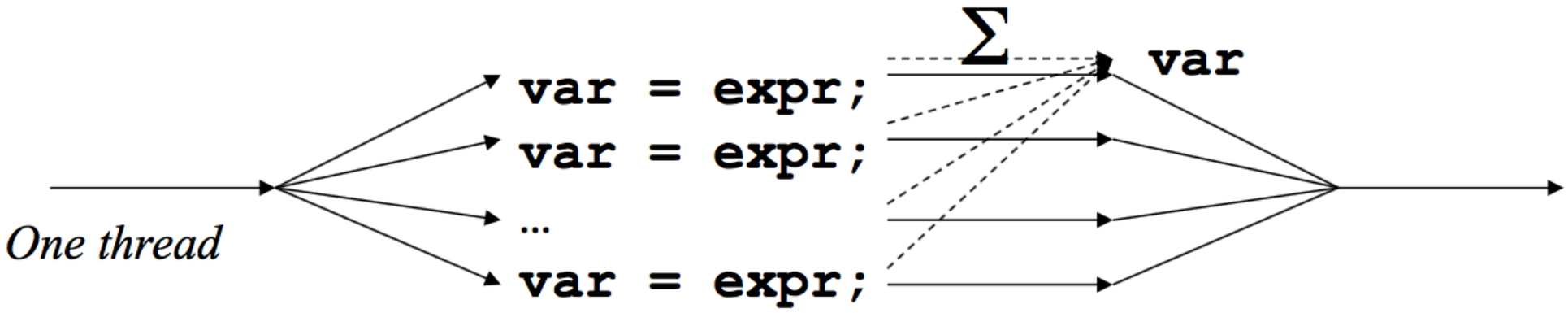

  ```

Parallel with reduction clause

- ```
  #pragma omp parallel reduction(+:var)
  {
     var = expr;
     ...
  }
  ... = var;
  ```

+, *, -, &, ^, |, &&, ||

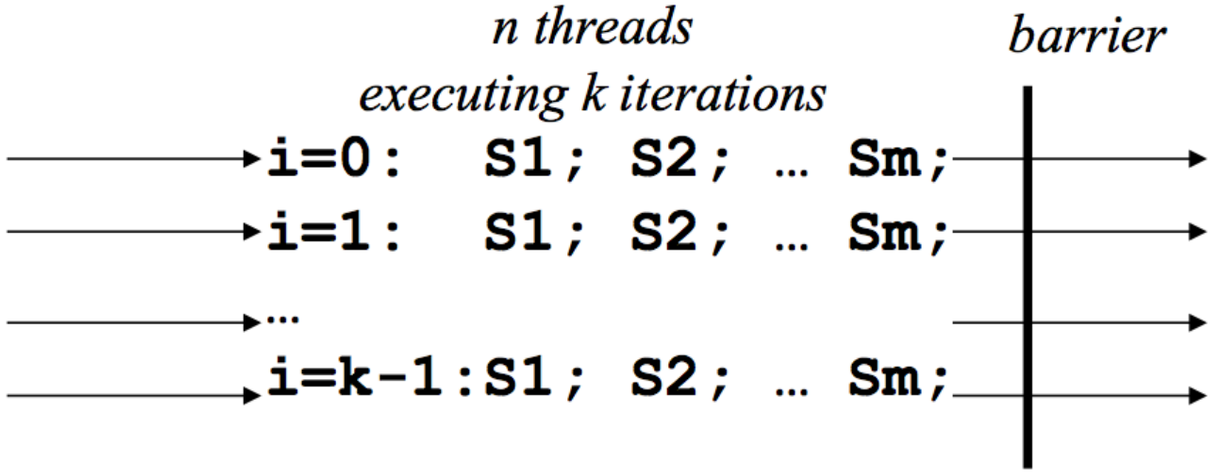
- Performs a global reduction operation over privatized variable(s) and assigns final value to master's private variable(s) or to the shared variable(s) when shared


  ```

for loop

- An existing team of threads is used to execute a loop concurrently
- Loop iterations are executed concurrently by *n* threads
  Use nowait clause to remove the implicit barrier.

```
#pragma omp parallel
. . .
#pragma omp for
for (i=0; i<k; i++) {
   S1;
   S2;
   . . .
   Sm;
}
```



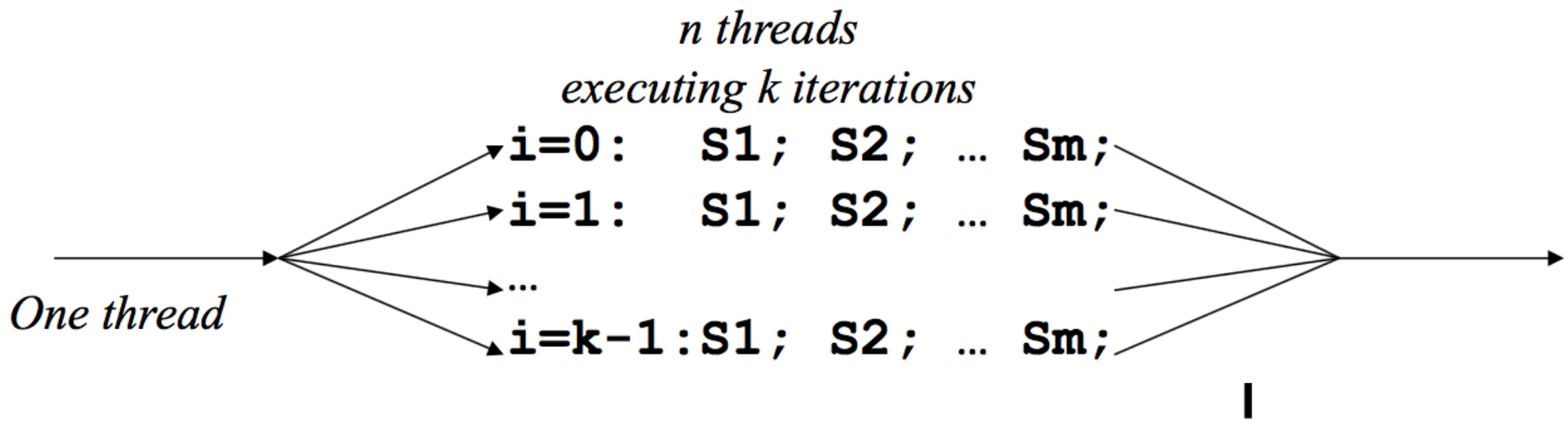
`for` loop

- An existing team of threads is used to execute a loop concurrently
- Loop iterations are executed concurrently by *n* threads
  Use no`wait` clause to remove the implicit barrier.

Simplify with:

```
#pragma omp parallel for
```

```
for (i=0; i<k; i++) {
   S1;
   S2;
   . . .
   Sm;
}
```

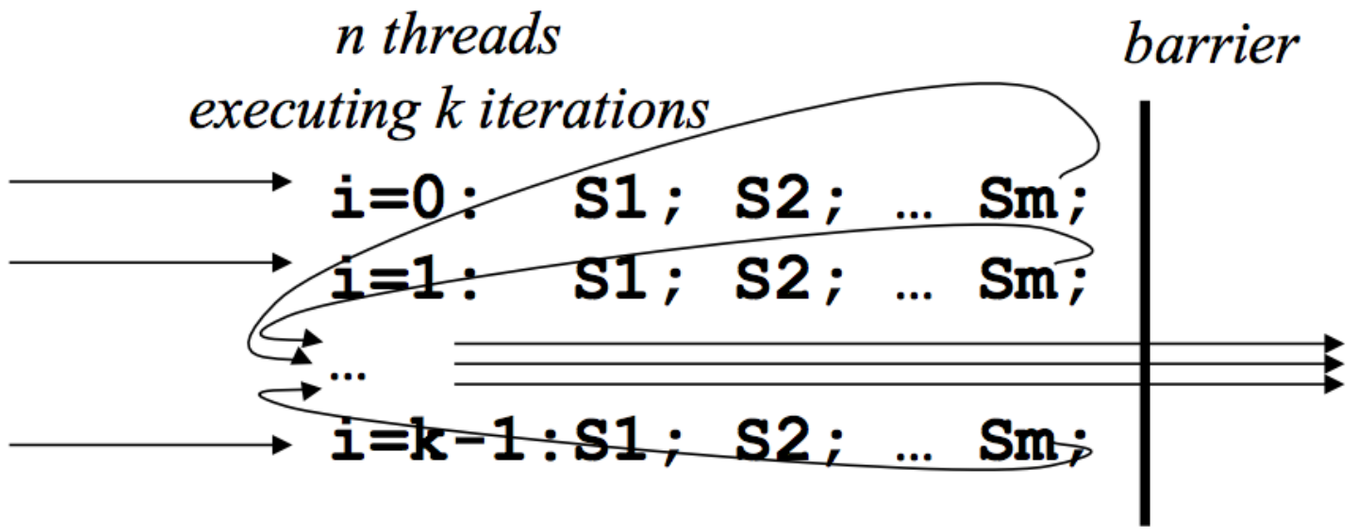


`for` loop scheduling

- Assignment of work loads to threads is controlled with a schedule clause, e.g.:

- `#pragma omp for schedule(dynamic)`

- When k>n, threads execute randomly chosen loop iterations until all iterations are completed



`for` loop scheduling

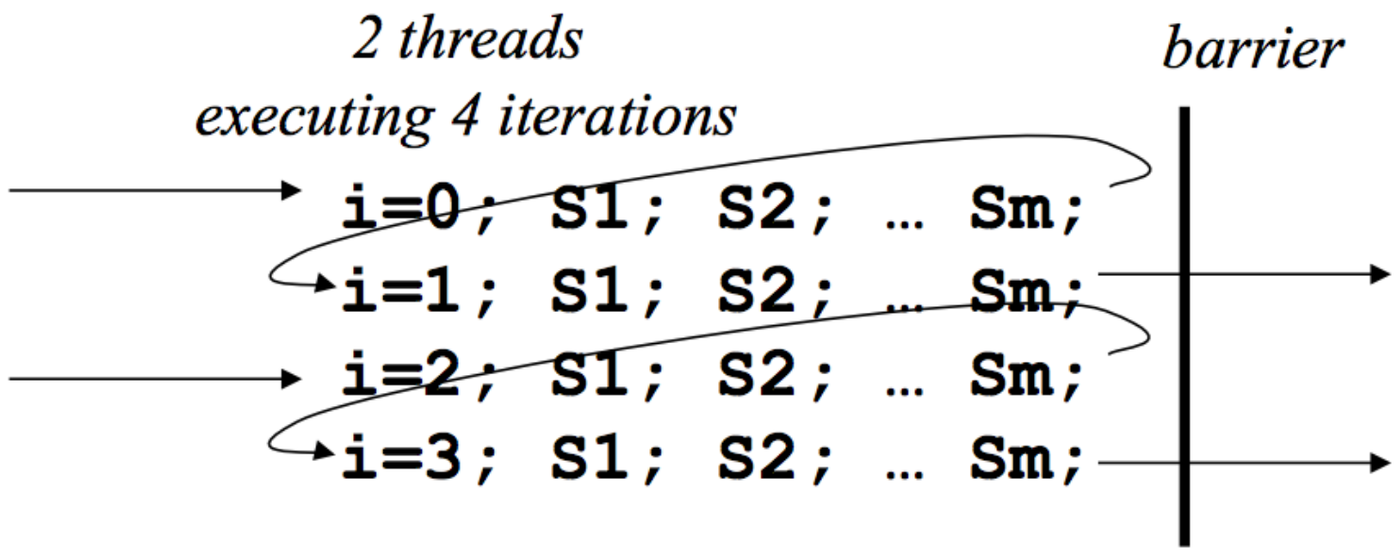
- Assignment of work loads to threads is controlled with a schedule clause, e.g.:

`for` loop scheduling

- Assignment of work loads to threads is controlled with a schedule clause, e.g.:

- `#pragma omp for schedule(static)`

- When k>n, threads are assigned to k/n chunks of the iteration space



Nested for loop

- If we use the for pragma to nested loops, by default the pragma applies only to the outer loop.

- We can of course insert another for pragma inside, to parallelize the inner loop.

- Starting with OpenMP version 3.0, one can use the collapse clause, e.g.

```
#pragma omp parallel for collapse(2)
```

  to specify two levels of nesting in the assignment of threads to tasks.

Nested for loop

The iterations of the inner loop must NOT depend on the outer loop.

According to the OpenMP specification under the description of the section on binding and the collapse clause:
“If execution of any associated loop changes any of the values used to compute any of the iteration counts, then the behavior is unspecified.”

- Starting with OpenMP version 3.0, one can use the collapse clause, e.g.

```
#pragma omp parallel for collapse(2)
```

to specify two levels of nesting in the assignment of threads to tasks.

Collapse directive

- The collapse-clause increases the total number of iterations that will be partitioned across the available number of OMP threads by reducing the granularity of work to be done by each thread.

- If the amount of work to be done by each thread is non-trivial (after collapsing is applied), this may improve the parallel scalability of the OMP application.
  It is only possible to collapse perfectly nested loops, that is, the loop body of the outer loop can consist only of the inner loop; there can be no statements before or after the inner loop in the loop body of the outer loop.

Example:

```
#pragma omp for
for ( i=0; i<N; i++ )
  for ( j=0; j<N; j++ )
    A[i][j] = B[i][j] + C[i][j]
```

- Modified example for better performance:

```
#pragma omp for collapse(2)
for ( i=0; i<N; i++ )
  for ( j=0; j<N; j++ )
    A[i][j] = B[i][j] + C[i][j]
```

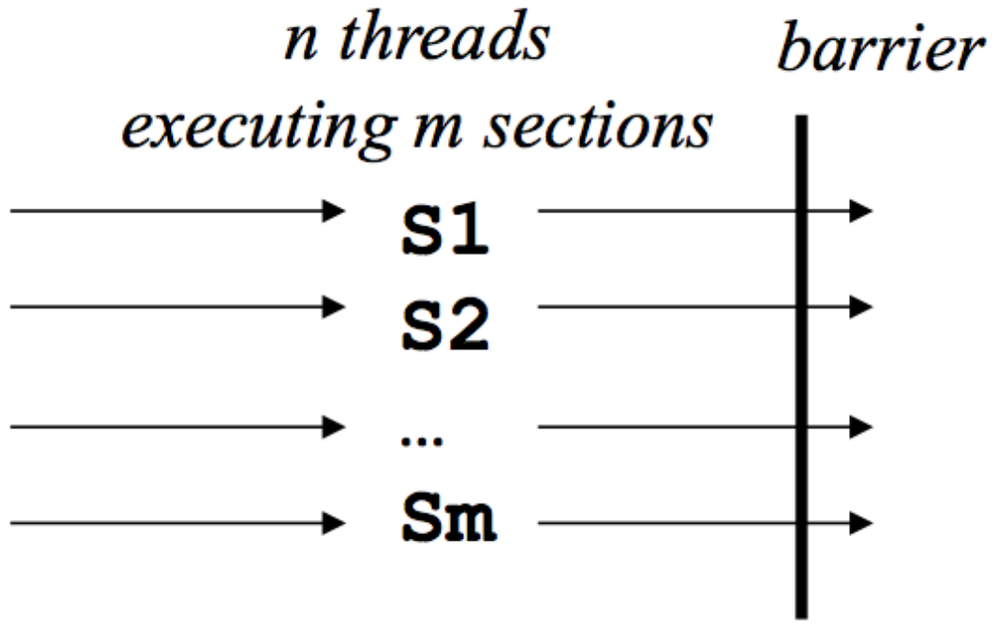
In general, the more work there is to divide over a number of threads, the more efficient the parallelization will be.

sections

- The sections construct is for work-sharing, where a current team of threads is used to execute statements of each section concurrently

```
#pragma omp parallel
. . .
#pragma omp sections
{
  #pragma omp section
  S1;

  #pragma omp section
  S2;
  . . .
  #pragma omp section
  Sm;
}
```



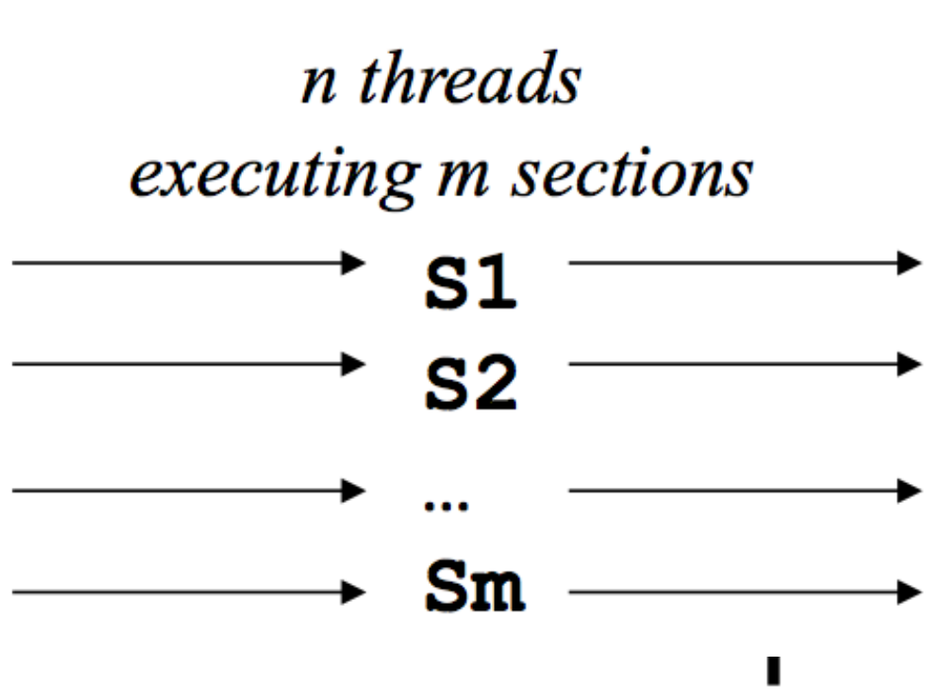
sections

- The sections construct is for work-sharing, where a current team of threads is used to execute statements of each section concurrently

```
#pragma omp parallel
. . .
#pragma omp sections
{
  #pragma omp section
  S1;

  #pragma omp section
  S2;
  . . .
  #pragma omp section
  Sm;
}
```

Use a `nowait` clause to remove implicit barrier

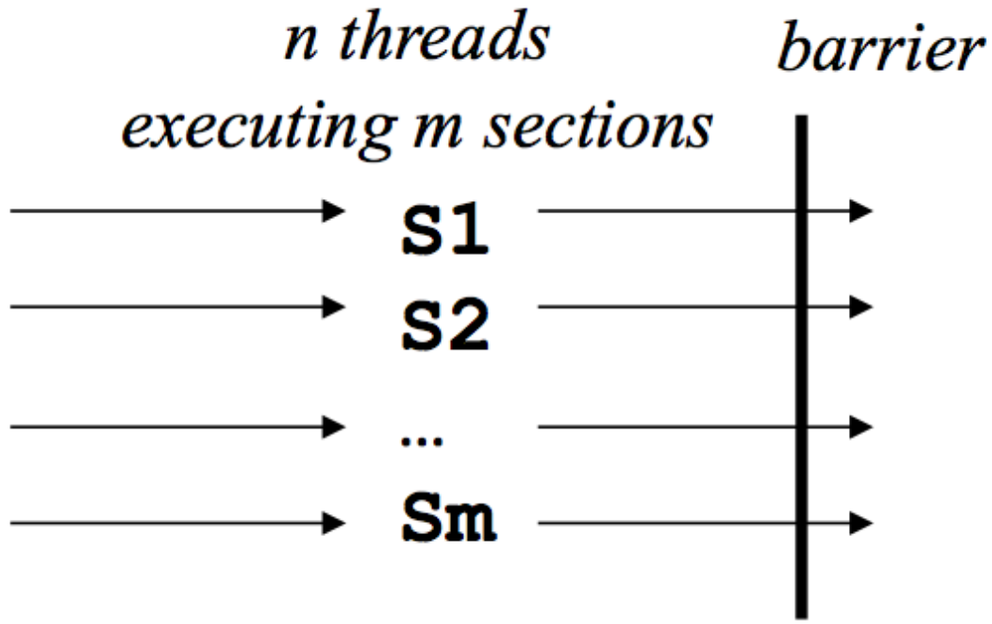


sections

- The sections construct is for work-sharing, where a current team of threads is used to execute statements of each section concurrently

```
#pragma omp parallel
. . .
#pragma omp sections
{
  #pragma omp section
  S1;

  #pragma omp section
  S2;
  . . .
  #pragma omp section
  Sm;
}
```



sections

- The sections construct is for work-sharing, where a current team of threads is used to execute statements of each section concurrently

```
#pragma omp parallel
. . .
#pragma omp sections
{
  #pragma omp section
  S1;

  #pragma omp section
  S2;
  . . .
  #pragma omp section
  Sm;
}
```

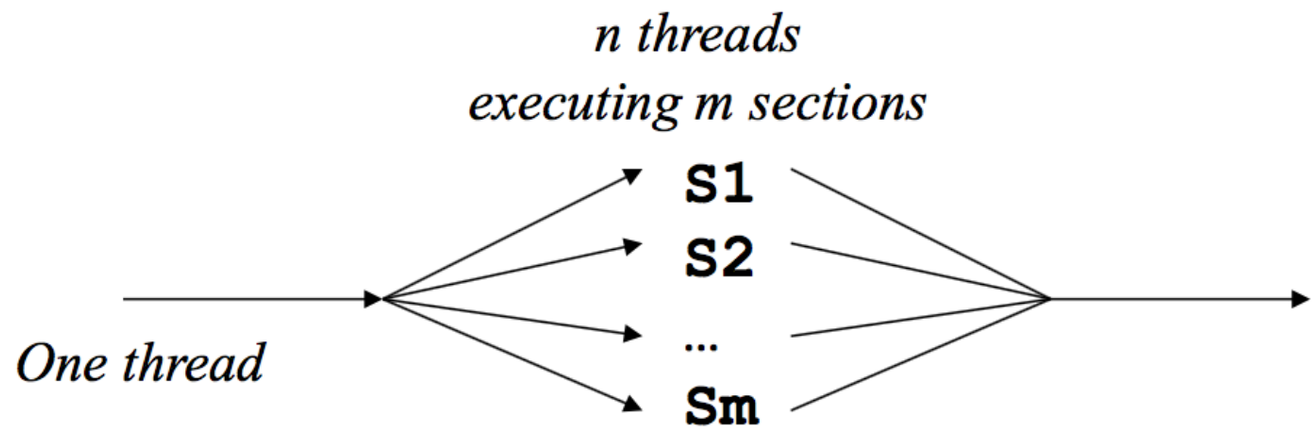
sections

- The sections construct is for work-sharing, where a current team of threads is used to execute statements of each section concurrently

Simplify with:
`#pragma omp parallel sections`

```
{
  #pragma omp section
  S1;

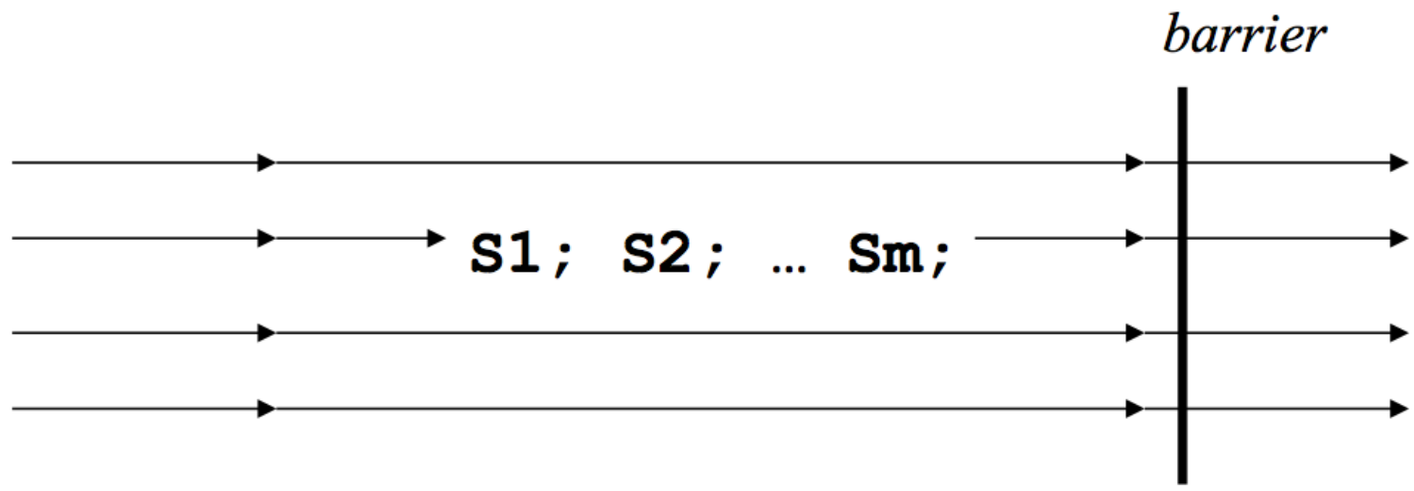
  #pragma omp section
  S2;
  . . .
  #pragma omp section
  Sm;
}
```



`single` execution

- The single construct selects one thread of the current team of threads to execute the body. Implicit barrier can be eliminated with nowait.

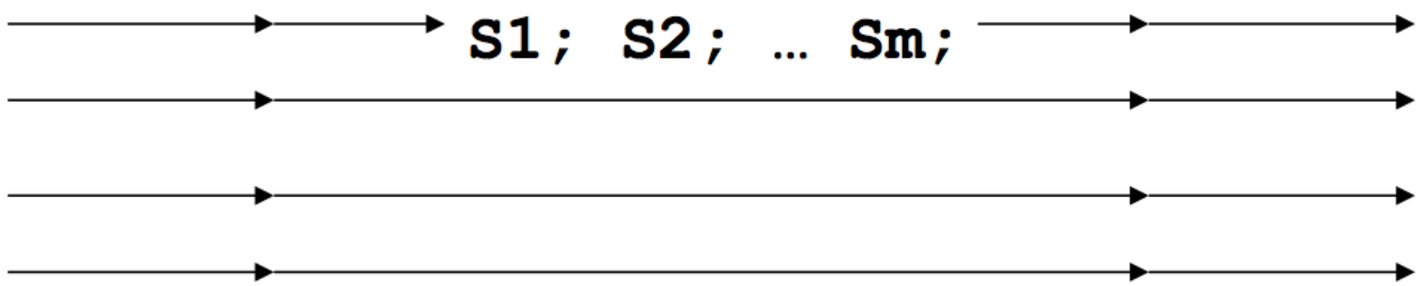
```
#pragma omp parallel
. . .
#pragma omp single
{
   S1;
   S2;
   ...
   Sm;
}
```



`master` execution

- The master construct selects the master thread of the current team of threads to execute the body, no barrier is inserted

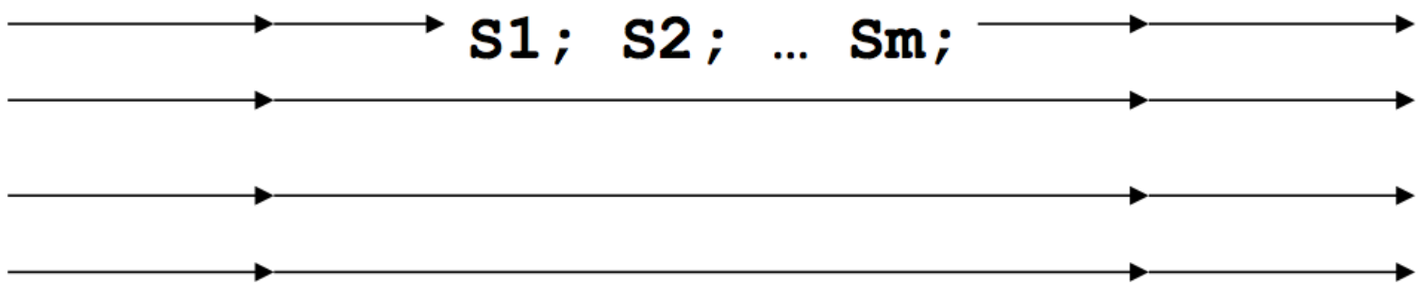
```
#pragma omp parallel
...
#pragma omp master
{
  S1;
  S2;
  ...
  Sm;
}
```



`master` execution

With the release of OpenMP standard v 5.1 in Nov. 2020, the term "`master`" was changed to "`masked`" to address concerns that it is offensive to many in the technical community. Use `#pragma omp masked` instead of `#pragma omp master`.
Compilers may take some time to implement this change.

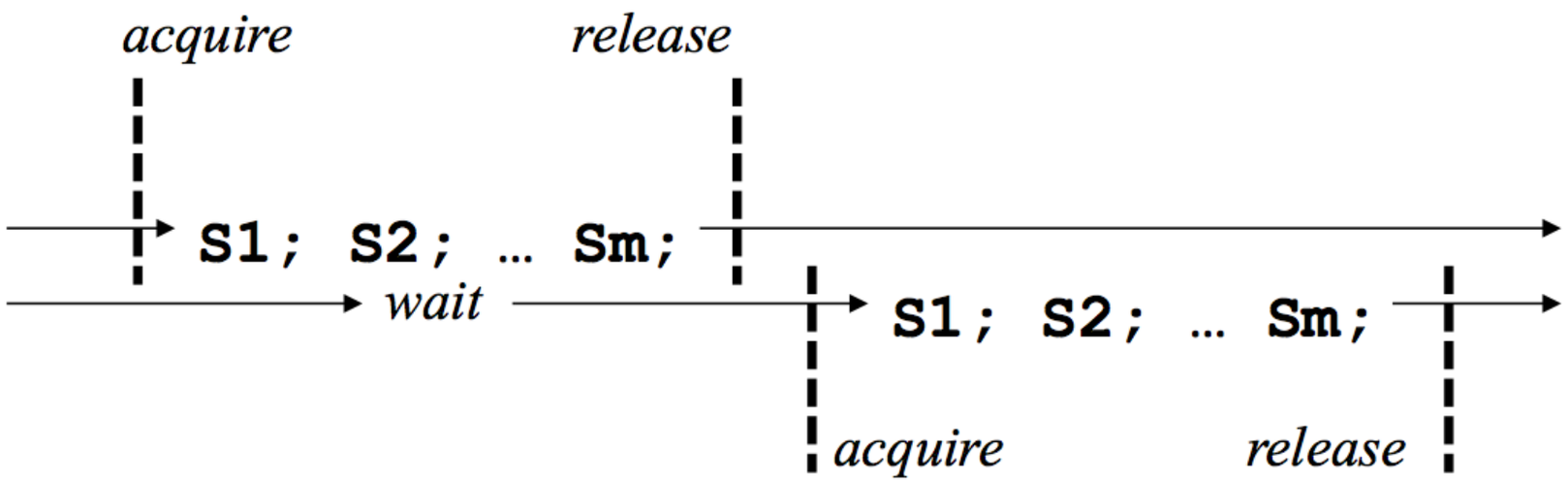
```
#pragma omp parallel
...
#pragma omp master
{
   S1;
   S2;
   ...
   Sm;
}
```



critical section

- The critical construct defines a critical section. Mutual exclusion is enforced on the body using a (named) lock

```
#pragma omp parallel
. . .
#pragma omp critical [name]
{
   S1;
   S2;
   . . .
   Sm;
}
```

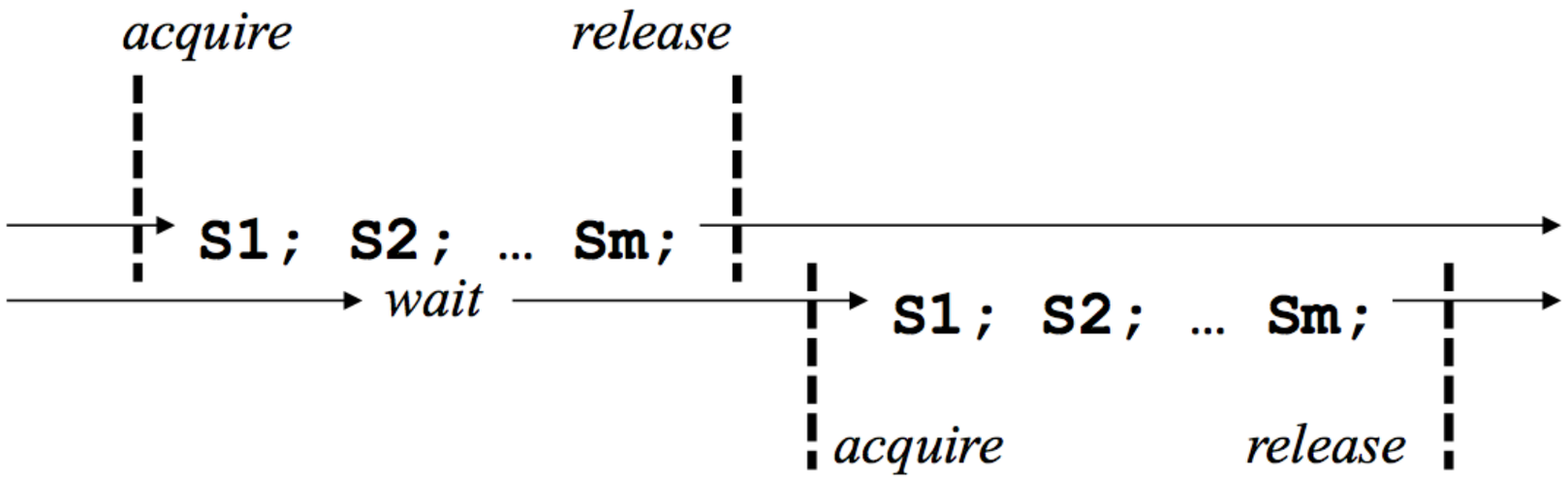


`critical` section

- The critical construct defines a critical section. Mutual exclusion is enforced on the body using a (named) lock

Two or more blocks protected with critical directives with different names can be executed simultaneously.

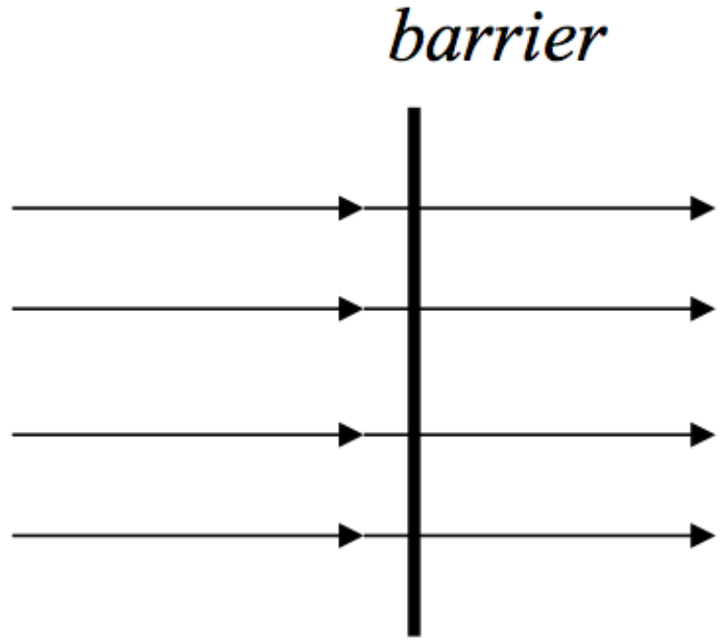
```
#pragma omp critical [name]
{
   S1;
   S2;
   . . .
   Sm;
}
```



`barrier` synchronization

- The barrier construct synchronizes the current team of threads

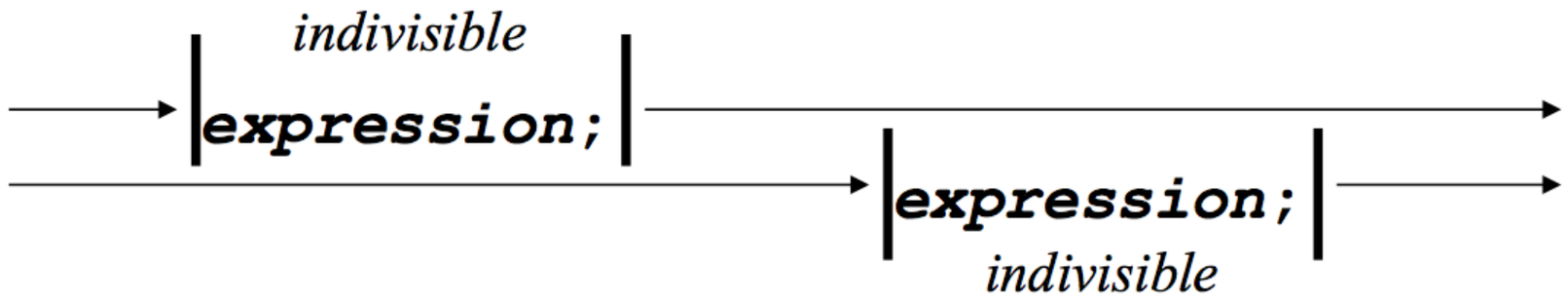
```
#pragma omp parallel
{
...
#pragma omp barrier
}
```



atomic execution

- The atomic construct executes an expression atomically (expressions are restricted to simple updates)

```
#pragma omp parallel
...
#pragma omp atomic
expression;
```



Locking

- Mutex locks, with additional “nestable” versions of locks that can be locked multiple times by the same thread

```
omp_lock_t lck;

omp_init_lock(&lck);
omp_set_lock(&lck);
...
... critical section ...
...
omp_unset_lock(&lck);
omp_destroy_lock(&lck);
```

Credits

- These slides report material from:
  - Prof. Robert van Engelen (Florida State University)
  - Prof. Jan Lemeire (Vrjie Universiteit Brussel)
  - Prof. Robert M. Keller (Harvey Mudd College)
  - Prof. Vivek Sarkar (Rice University)
  - Jay P. Hoeflinger (Intel) and Bronis R. de Supinski (LLNL)

Books

- The Art of Concurrency, Clay Breshears, O'Reilly - Chapt. 5
- Principles of Parallel Programming, Calvin Lyn and Lawrence Snyder, Pearson - Chapt. 6
- Parallel Programming for Multicore and Cluster Systems, Thomas Dauber and Gudula Rünger, Springer - Chapt. 6
- An introduction to parallel programming, Peter S. Pacheco, Morgan Kaufman - Chapt. 5
- Parallel and High Performance Computing, R. Robey, Y. Zamora, Manning - Chapt. 7